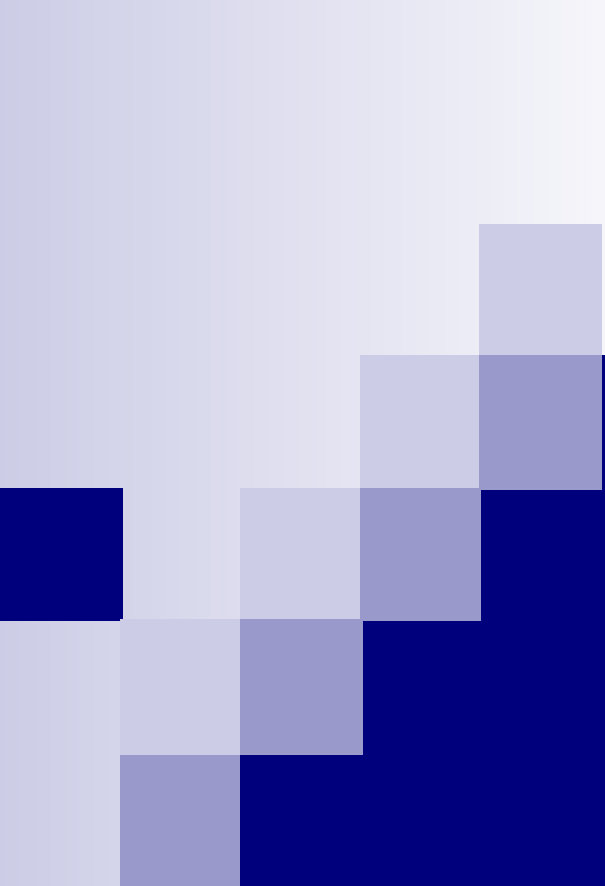




Circuitos Combinacionais Eletrônica Digital **Introdução**

Fernando Passold, Dr. Eng.

Profesor Titular III

Engenharia Elétrica

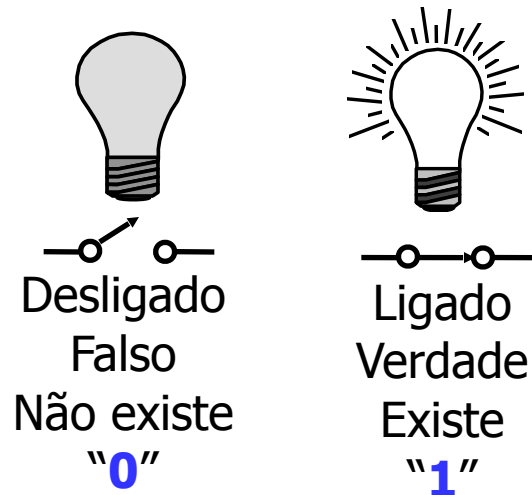
E-mail: fpassold@upf.br

Introdução

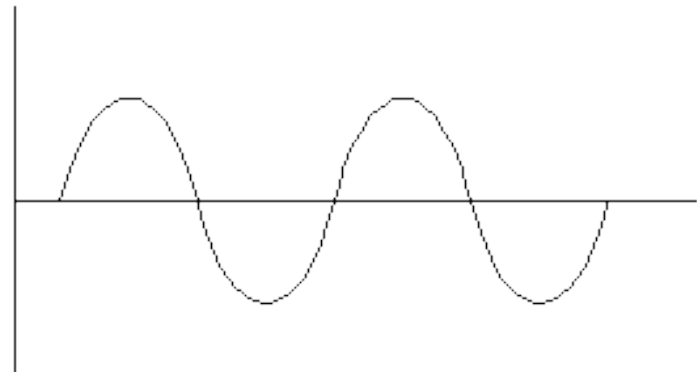
- Sistemas Numéricos
 - Códigos Digitais
 - Sistemas numéricos mais utilizados
 - Formação dos sistemas numéricos
 - Conversão entre diferentes bases numéricas
 - Outros códigos binários (Gray, ASCII)
- Álgebra de Boole
 - Portas Lógicas básicas (tabela verdade, símbolo, equações)
 - Análises
 - Propriedades de Álgebra de Boole
 - Minimização de funções algébricas (e de circuitos combinacionais digitais – 1ª parte)

Introdução

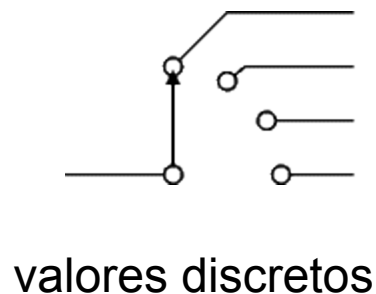
Mundo Digital



Mundo Analógico



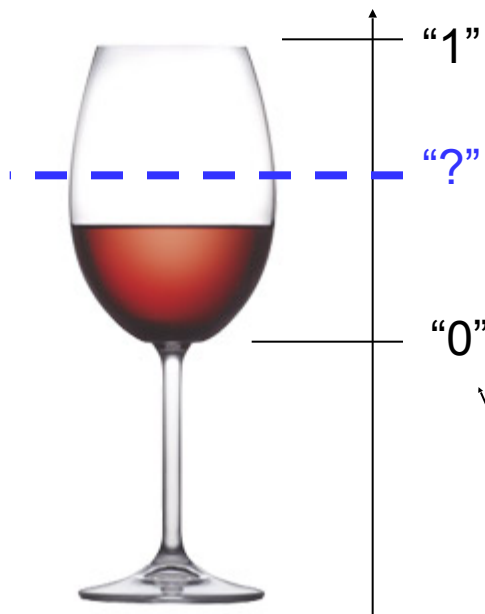
Semáforo



Infinitos estados;
Qualquer valor;
Valor em qualquer instante de tempo.

Introdução

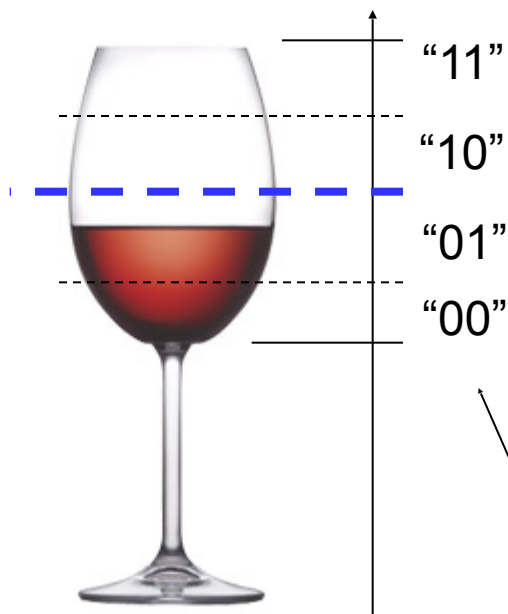
- Estados binários: “0” ou “1”:



Note: usando apenas 1 bit!

Introdução

- Estados binários: “0” ou “1”:

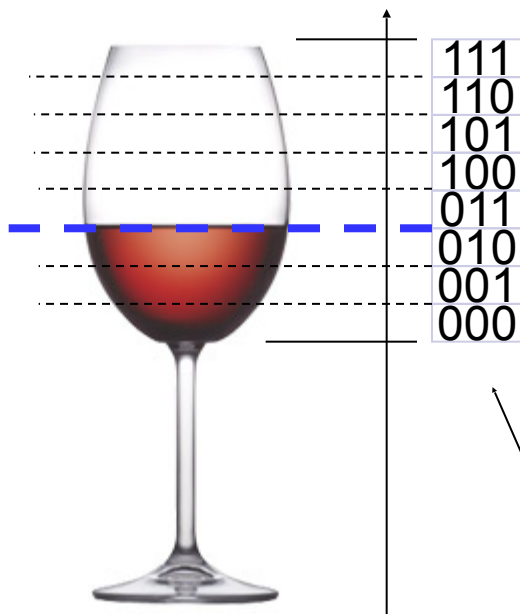


- Processo de Digitalização de Sinais:
- Problemas de Quantização y Codificação

Note: usando 2 bits!

Introdução

- Estados binários: "0" ou "1":

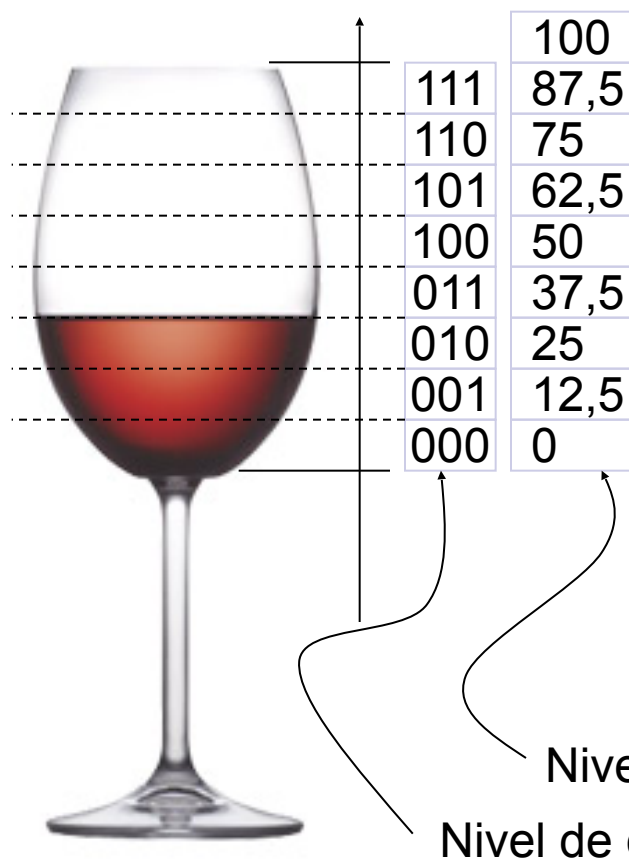


- 3 bits $\rightarrow 2^3 = 8$ valores diferentes;
- Faixa de excursão: 0 ~ 100%:
- Escala de quantização, ou "resolução": $100/8 = 12,5$

Note: usando 3 bits!

Introdução

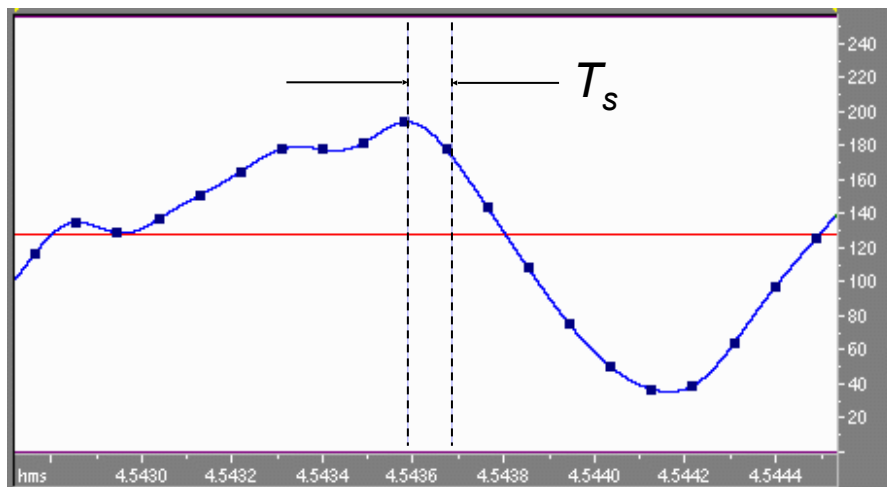
- Estados binários: "0" ou "1":



- 3 bits $\rightarrow 2^3 = 8$ valores diferentes;
- Faixa de excursão: 0 ~ 100%;
- Escala de quantização, ou "resolução": $100/8 = 12,5$
- Se o valor real é 35 $\rightarrow$ erro de: -10 o +2,5 (depende do nível de decisão).

Introdução

- Como quantizar valores de tensão negativos ?
 - Também existem várias formas.
- O exemplo seguinte mostra o caso para arquivos digitais de áudio em formato ***.WAV com 8 bits**:

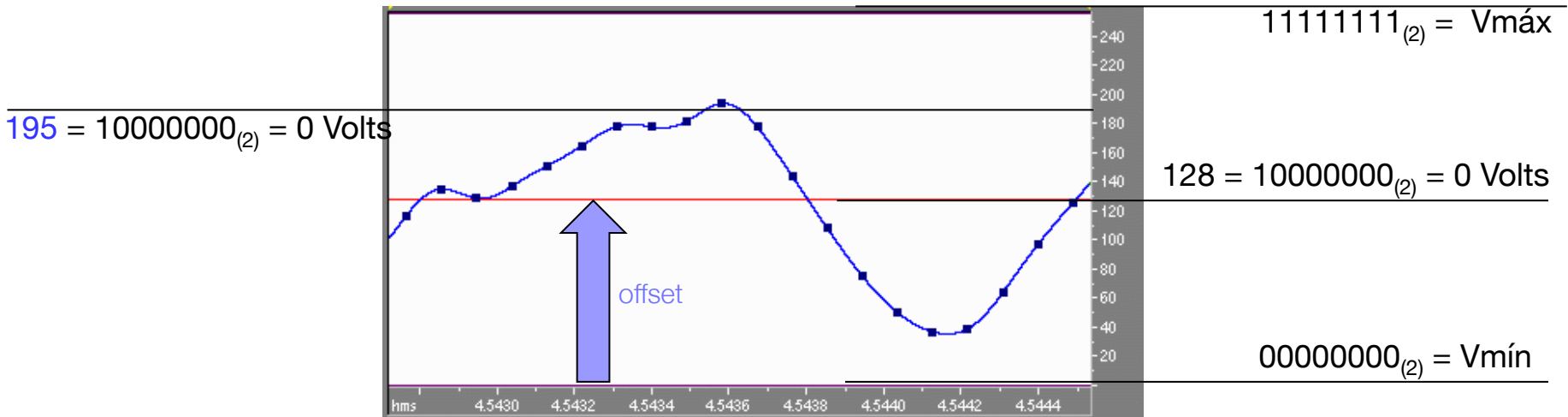


Note: o sinal é “fatiado” (**amostrado** ou **discretizado**) no tempo.

T_s : período de amostragem.

Obs.: O teorema de amostragem de Nyquist ou Shanon determina valores adequados para T_s (não estudado nesta disciplina, faltam fundamentos de Laplace e Diagramas Espectrais).

Arquivo digital de áudio em formato *.WAV com 8 bits:

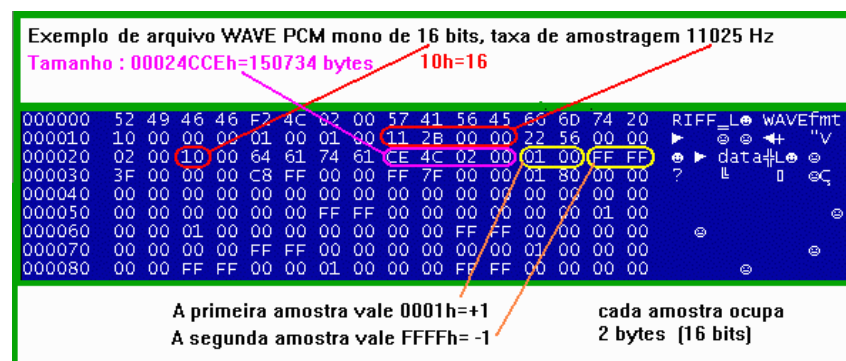
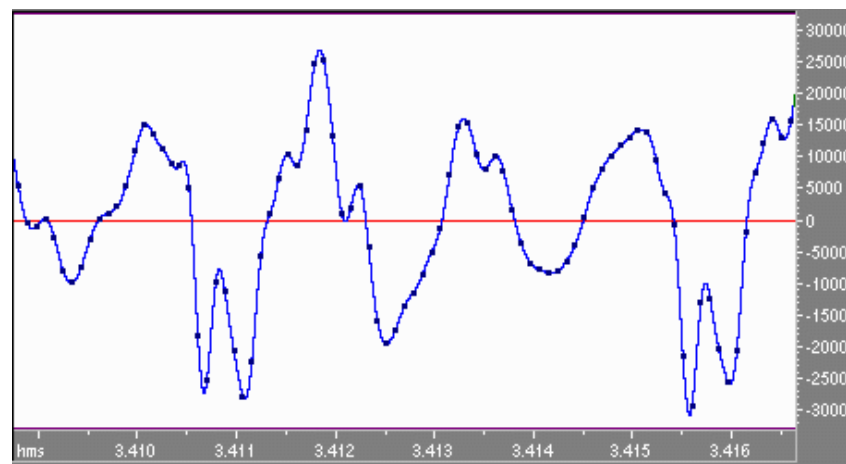


- O eixo vertical da figura é graduado no valor das amostras quantizadas com 8 bits : 0 a 255 (2^8).
- Note que o eixo de tensão, 0 Volts, sofre um “**offset**” (deslocamento) para o código (número): 128.
 - Pode-se assim representar valores negativos de tensão sem necessidade de usar código binário com sinal. A forma de onda quantizada acima, no formato decimal é :
118,135,130,138,151,165,179,179,182,195,179,144,109,78,51,37,39,62,97,123. (← unsigned char em “C”)
- O que representa os seguintes valores quantizados de tensão (em Volts), supondo que V_{máx}=255 Volt e que V_{mín}= Volts:
 - -10,+7,+2,+10,+23,+37,+51,+51,+54,+67,+51,+16,-19,-50,-77,-91,-89,-66,-31,-5 (Volts)

Isto é:
 $255_{(10)} = FF_h \rightarrow 255 \text{ Volts}$
 Cada “1” $\rightarrow$ 1 Volt, mas... Não esquecer “offset”, então:

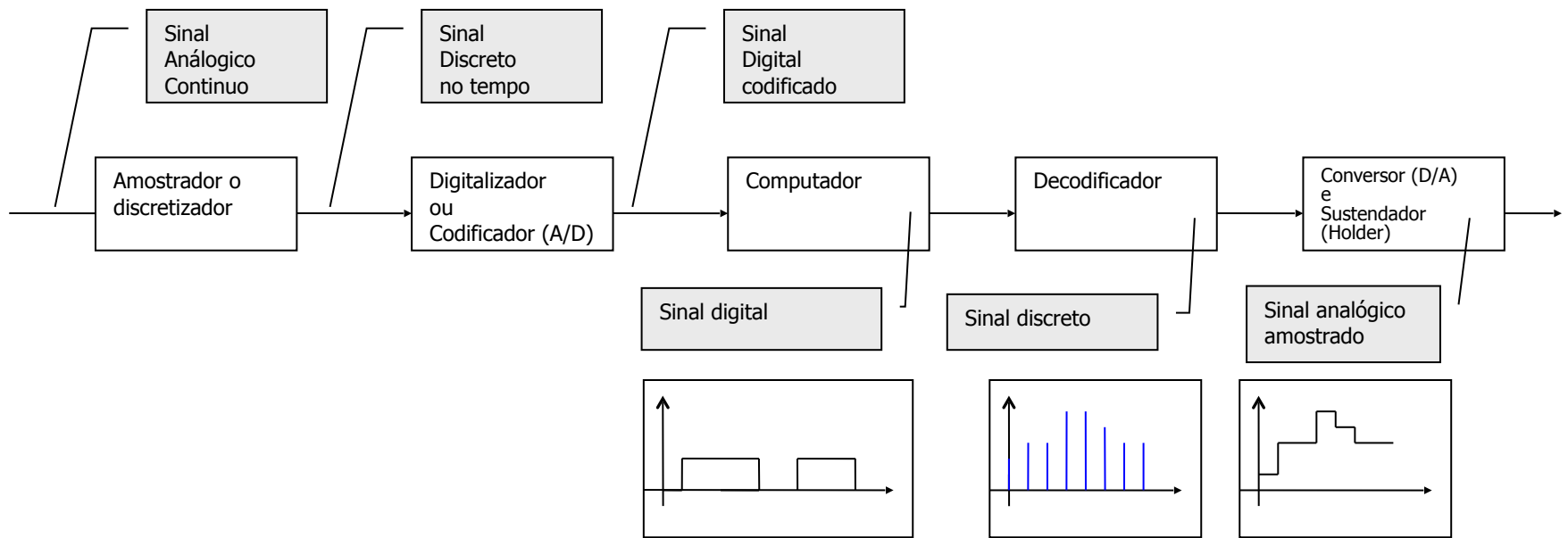
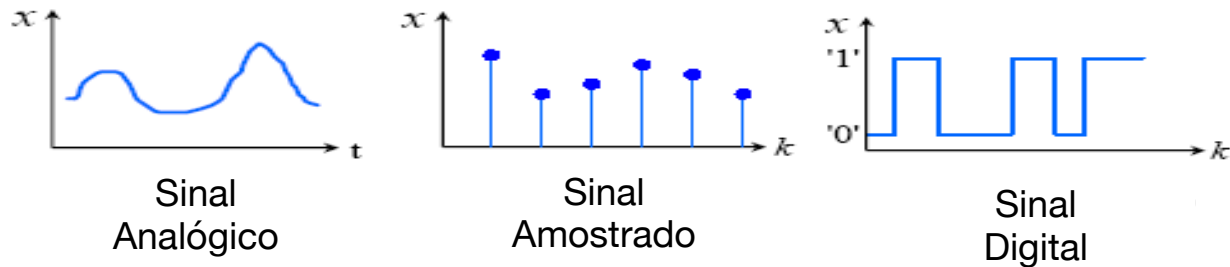
Arquivo de áudio digital PCM em formato ***.WAV de 16 bits**

- Um arquivo de áudio digital PCM no formato ***.WAV de 16 bits** usa condificação com **senal-complemento de 2**.
- Valores positivos são codificados de 0000h=0 até 7FFFh=+32767 e valores negativos são codificados de FFFFh=-1 até 8001h=-32767. O zero é codificado como: 0000H=0.
- A primeira figura representa esta codificação (eixo vertical):
- A segunda figura representa a parte inicial de um arquivo *.WAV de 16 bits.



Obs.: Cada Conjunto de 8-bits => 1 byte.

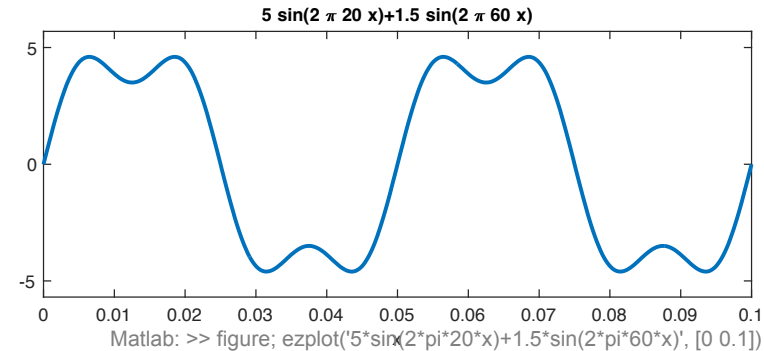
Digitalização de um sinal...



Sinais Contínuos x Discretos

- Sinal Analógico:

- Ex.: $y(t) = 5 \sin(2\pi 20t) + 1,5 \sin(2\pi 60t)$



- Eletrônica analógica: transístores na faixa linear, β .

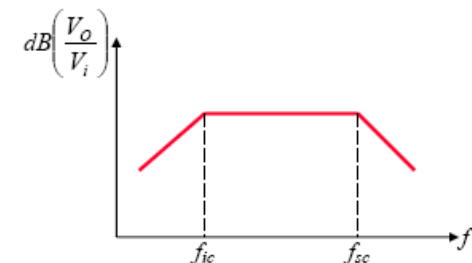
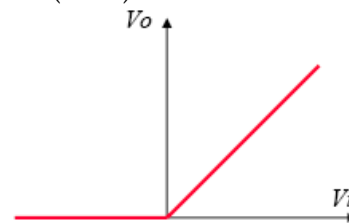
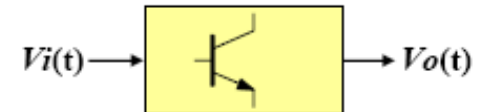
- Modelo matemático (forma de caracterizar):

- Função transferência: $V_o(t) = f(V_i(t))$

- Equação diferencial (modelo do circuito).

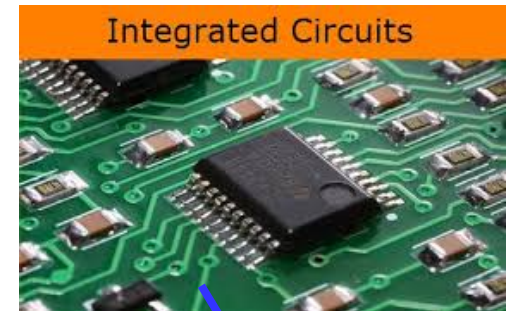
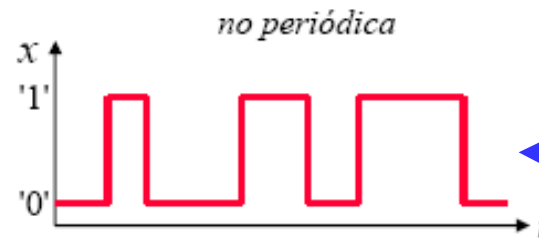
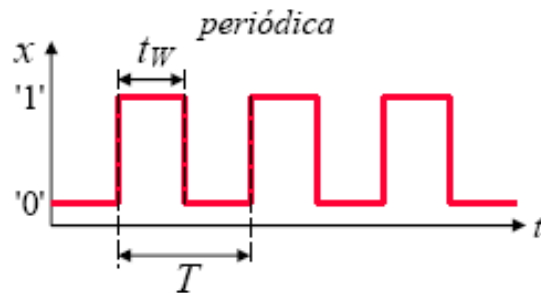
- Função de transferência no plano-s: $\frac{V_o(s)}{V_i(s)} = H(s)$

- Resposta em frequência: $|H(j\omega)|$



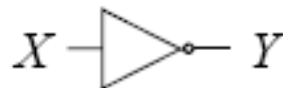
Sinais Contínuos x Discretos

- Sinal digital (binário): trem de pulsos:

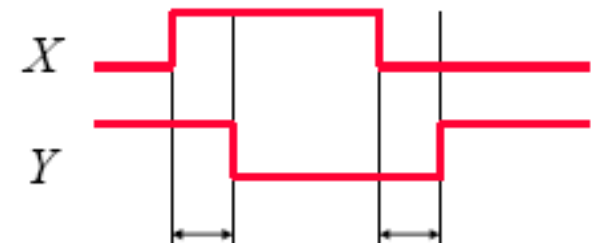


Cada trilha

- Transistor $\rightarrow$ interruptor (corte/saturação)
- Caracterização:
 - Tabelas verdade
 - Diagramas de transição

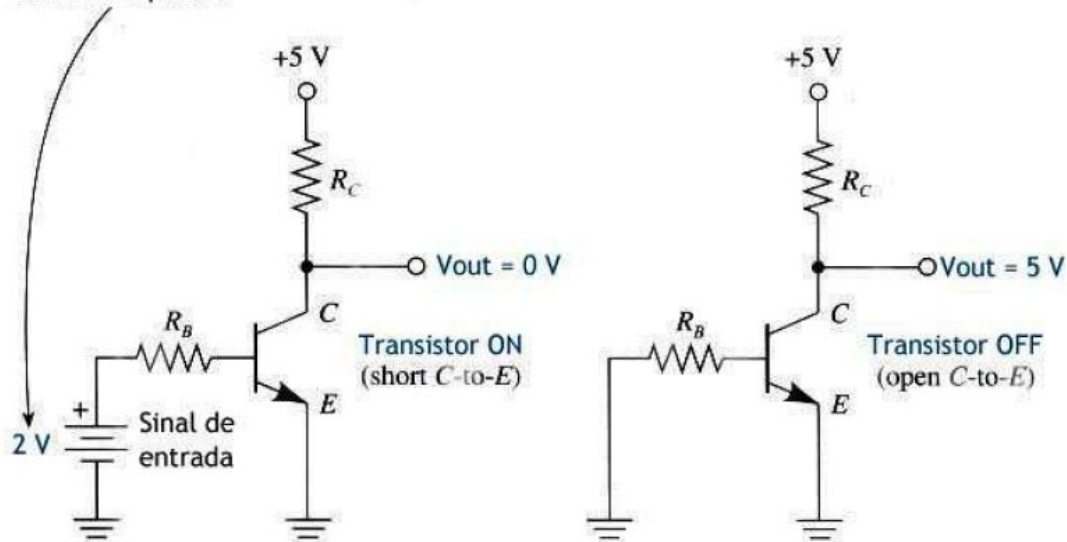


X	Y
0	1
1	0



Transistor operando como interruptor

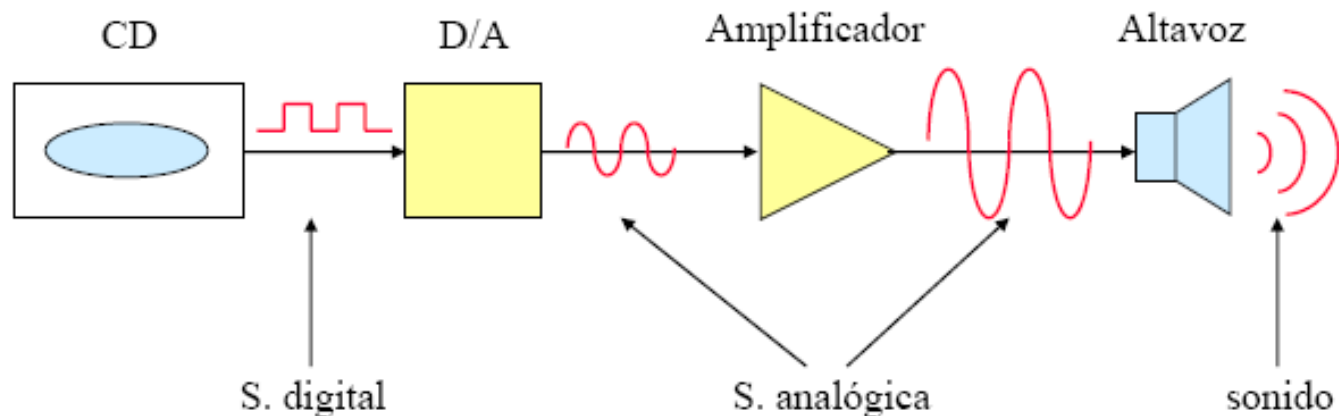
Uma tensão positiva na base do transistor NPN causa um curto de C para E.



1. Em um transistor PNP, se uma tensão positiva for aplicada entre sua base e o emissor, a junção coletor-emissor torna-se um circuito fechado (pode-se dizer que o "transistor foi ligado" ou que está "saturado").
2. Aplicar uma tensão negativa ou nula (0 V) entre a base e o emissor, abre a junção coletor-emissor (pode-se dizer que o "transistor foi desligado" ou que está em estado de "corte").

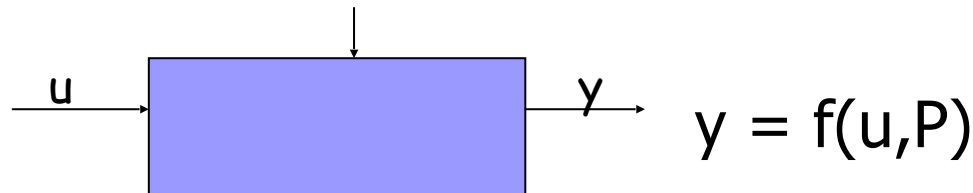
Sistema misto (analógico-digital)

- Circuitos de conversão:
 - Conversores A/D e D/A.
- Exemplo:
 - Reprodutor de CD
 - Placa de áudio de PC



Sistemas Digitais.

- A teoria de Sistemas permite descrever um sistema através de diagramas, o que permite ver suas diferentes partes, para formular um modelo matemático que relaciona 3 tipos de variables: entradas, saídas y perturbações.



- No caso de Sistemas Digitais, sistemas criados pelo homem, os modelos se aproximam bastante mais que em outros sistemas físicos reais.

Vantagens dos sistemas Digitais

- 1. Sistemas digitais são geralmente mais fáceis de projectar.
 - Porque os circuitos utilizados são circuitos de comutação ($0 \rightleftharpoons 1$), onde o importante não é tratar de forma exata os valores de tensão e corrente (como em eletrônica analógica), mas apenas a faixa que alcançam (nível lógico “Alto” ou “Baixo”: “HIGH” or “LOW”).
- 2. Guardar (estocar) informação é (bem) mas fácil.
 - Isso é possível graças a circuitos de comutação especiais (biestáveis) que podem guardar informação e mantê-las por tanto tempo quanto seja necessário (sistemas digitais sequenciais: digitais II). Em alguns casos, necessitando energia (memórias voláteis), em outros nem isso.
- 3. Exatidão e precisão melhores.
 - Sistemas digitais podem manipular tantos dígitos de precisão quanto se necessite simplesmente se adicionando mais circuitos de comutação (aumentando os bits). Em sistemas analógicos, a precisão está geralmente limitada a 3 ou 4 dígitos porque os valores de tensão e corrente dependem diretamente (da qualidade e) dos componentes do circuito e estes ainda são afetados por perturbações aleatórias (ruído, temperatura $\rightarrow$ “drift”).

Sistemas numéricos binários mais usados:

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F						
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15						
binário		octal																			
decimal																					
hexadecimal																					

- Base **binária**: 2 símbolos: "0" e "1"
- Base **decimal**: 10 símbolos: 0, 1, 2, 3, 4, 5, 6, 7, 8 e 9.
- Base **octal**: 8 símbolos: 0, 1, 2, 3, 4, 5, 6, 7.
- Base **hexadecimal**: 16 símbolos: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F.

Formação de um sistema numérico

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F								
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15								
binaria		octal																					
decimal																							
hexadecimal																							

$$N_b = a_{m-1} \times b^{m-1} + \boxed{?} + a_0 \times b^0 + a_{-1} \times b^{-1} + \boxed{?} + a_{-n} \times b^{-n}$$

$$N_b = \sum_{i=-n}^{m-1} a_i \times b^i$$

Onde:

N_b = número na base deseada;

a = símbolo que compõe o número na base desejada;

m e n = expoentes, índices, ou “posição” do símbolo dentro do número que está sendo formando.

Formação de um sistema numérico

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F														
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15														
binaria		octal																											
																decimal													
																								hexadecimal					

$$N_b = a_{m-1} \times b^{m-1} + \boxed{?} + a_0 \times b^0 + a_{-1} \times b^{-1} + \boxed{?} + a_{-n} \times b^{-n}$$

- Ex₁) $1042_{(10)} = 1 \times 10^3 + 4 \times 10^1 + 2 \times 10^0$

- Ex₂) $176_{(8)} = 1 \times 8^2 + 7 \times 8^1 + 6 \times 8^0$

$$= 64 + 56 + 6$$

$$= 126$$

Formação de um sistema numérico

- Ex₃: $100110_2 \rightarrow ?_{10}$

Utilizando a equação geral:

$$N_b = \sum_{i=-n}^{m-1} a_i \times b^i$$

5 4 3 2 1 0 ← “posição” (“índice”)

1	0	0	1	1	0
---	---	---	---	---	---

(2)

$$\begin{aligned} &= 1 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \\ &= 32 + 0 + 0 + 4 + 2 + 0 \\ &= 38_{(10)} \end{aligned}$$

Formação de um sistema numérico

$$N_b = \sum_{i=-n}^{m-1} a_i \times b^i$$

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F														
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15														
binaria		octal																											
																decimal													
																								hexadecimal					

■ Ex₄:

3 2 1 0, -1 -2 -3

$$\begin{aligned}
 1011,011_{(2)} &= 1 \times 2^3 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-2} + 1 \times 2^{-3} \\
 &= 8 + 2 + 1 + 0,25 + 0,125 \\
 &= 11,375
 \end{aligned}$$

Formação de um sistema numérico

$$N_b = \sum_{i=-n}^{m-1} a_i \times b^i$$

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F						
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15						
binaria		octal																			
								decimal													
										hexadecimal											

- Ex₅: 453₍₅₎ = ?
- Ex₆: 3456H = 3456₍₁₆₎ = ?₍₁₀₎ ← unsigned char a=0x3456;

Formação de um sistema numérico

$$N_b = \sum_{i=-n}^{m-1} a_i \times b^i$$

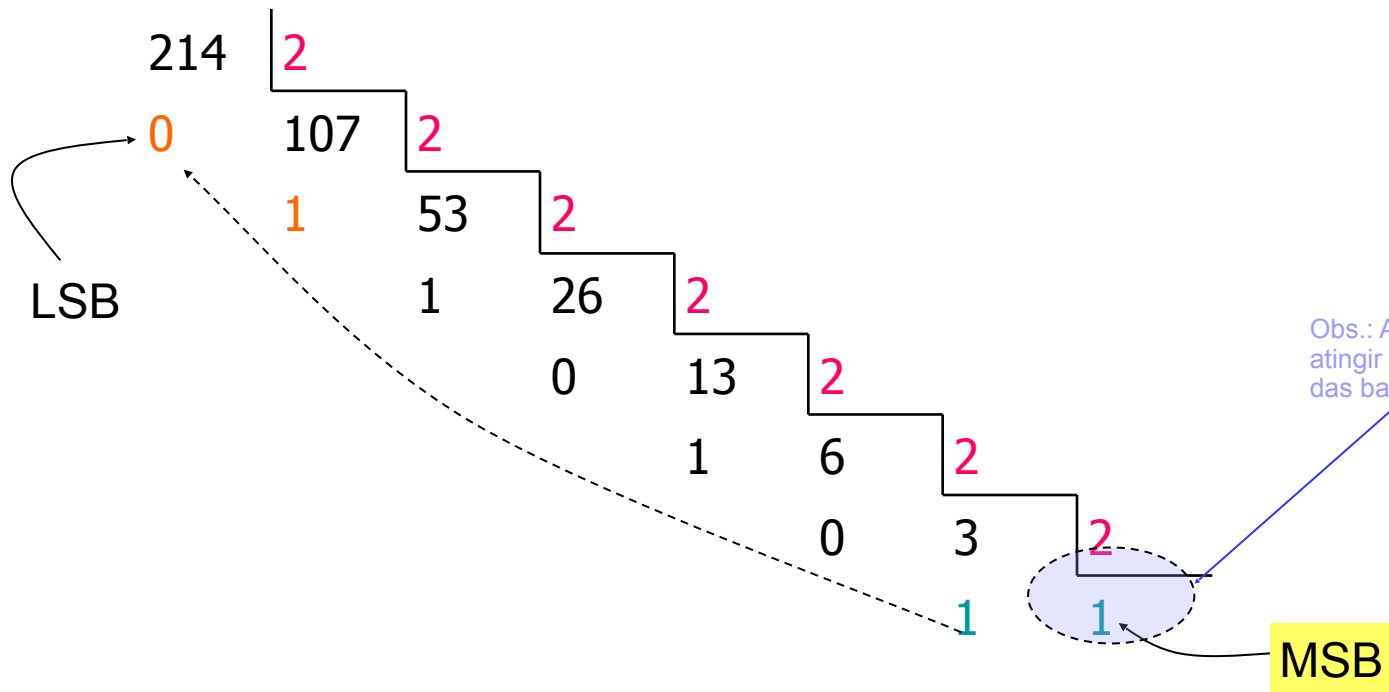
0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F						
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15						
binaria		octal																			
								decimal													
										hexadecimal											

- Ex₅: 453₍₅₎ = ?
- Ex₆: 3456H = 3456₍₁₆₎ = ?₍₁₀₎ ← unsigned char a=0x3456;
Resposta: 13398₍₁₀₎

Conversão de Códigos

- Método da divisão:

□ Ex₇: $214_{(10)} = ?_{(2)} = 11010110_{(2)}$



$$N_b = \sum_{i=-n}^{m-1} a_i \times b^i$$

Obs.: A divisão continua até o resultado atingir um número dentro dos símbolos das base de destino

Conversão de Códigos

- Método de la división:

□ Ex₈: $214_{(10)} = ?_{(16)} = \text{D6H}$

$$\begin{array}{r} 214 \\ -208 \\ \hline 6 \end{array}$$

16
13
6

MSB

LSB

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F														
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15														
binaria		octal																											
decimal																													
hexadecimal																													

Conversão de Códigos

- Ex₉: $117_{(10)} = ?_{(5)} = 432_{(5)}$

Conversão de Códigos

- Outro método: por aproximações sucessivas...
- Exemplo: $113_{(10)} = ?_{(2)}$

Note:

$$2^4 = 16$$

$$2^5 = 32$$

$$2^6 = 64$$

$$2^7 = 128$$

⇒

$$113 - 2^6 = 49$$

$$49 - 2^5 = 17$$

$$17 - 2^4 = 1$$

$$1 \times 2^0$$

⇒

6	5	4	3	2	1	0
1	1	1	0	0	0	1

Conversão de Códigos

- De binário à octal

$$\begin{array}{ccccccccc} & \underline{10} & \underline{110} & \underline{010} & \underline{110} & \underline{101} & & & \\ = & 2 & 6 & 2 & 6 & 5 & & & \end{array} \begin{array}{l} (2) \\ (8) \end{array}$$

- De octal à binário

$$\begin{array}{ccccccc} & \underline{5} & \underline{2} & \underline{7} & \underline{3} & & \\ = & 101 & 010 & 111 & 011 & & \end{array} \begin{array}{l} (8) \\ (2) \end{array}$$

- De binário à hexadecimal

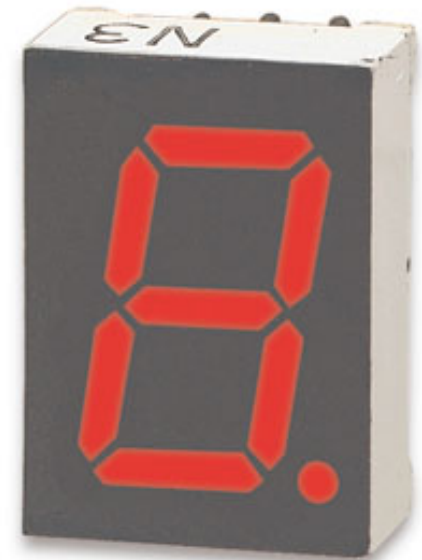
$$\begin{array}{ccccccccc} & \underline{1101} & \underline{0111} & \underline{1110} & \underline{0111} & \underline{0101} & & & \\ = & D & 7 & E & 7 & 5 & & & \end{array} \begin{array}{l} (2) \\ (16) \end{array}$$

Note: cada conjunto de 3 bits → 1 símbolo octal;
cada conjunto de 4 bits → 1 símbolo hexadecimal.

Código BCD (Binary Code Decimal)

Usa sempre 4 bits para representar os números:
0,1,2, ... , 8,9

BCD	decimal	BCD	decimal
0000	0	0101	5
0001	1	0110	6
0010	2	0111	7
0011	3	1000	8
0100	4	1001	9



7 Segments Display

Código BCD (Binary code decimal)

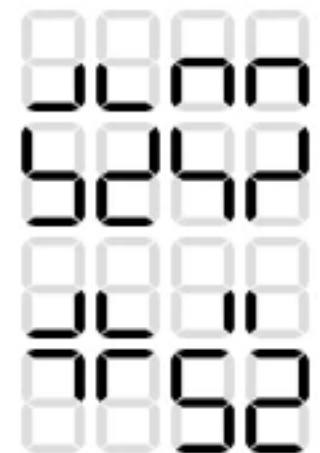
- Ex₁: Número 137₍₁₀₎ = ?_(2,BCD)

137₍₁₀₎ = 1000 1001 (binário)

137₍₁₀₎ = 0001 0011 0111 (BCD) *

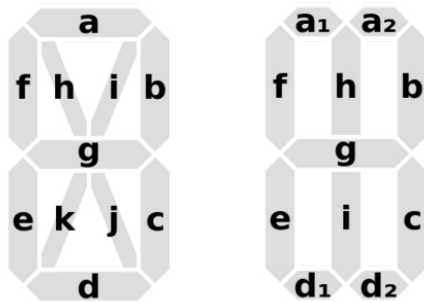
↓ ↓ ↓
1 3 7

* Note que o código BCD exigiu 12 bits (dígitos),
Enquanto que em binário puro teriam sido "gastos" somente 8 bits
para representar este número.

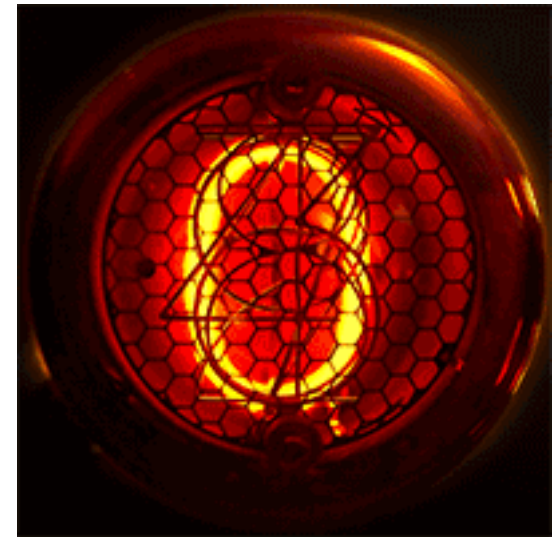


Outros Displays

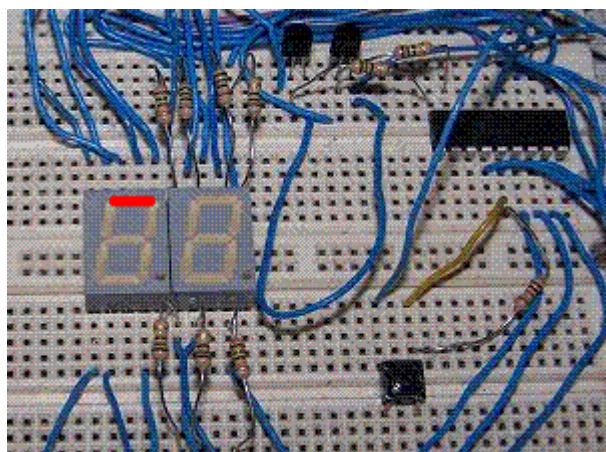
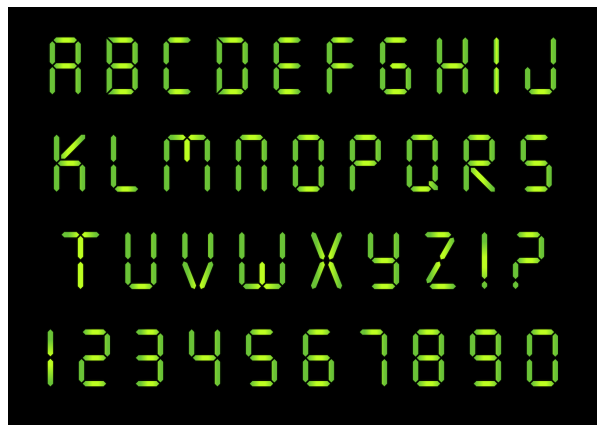
- 11 segmentos:



- 14 segmentos:



Outros usos para Displays:



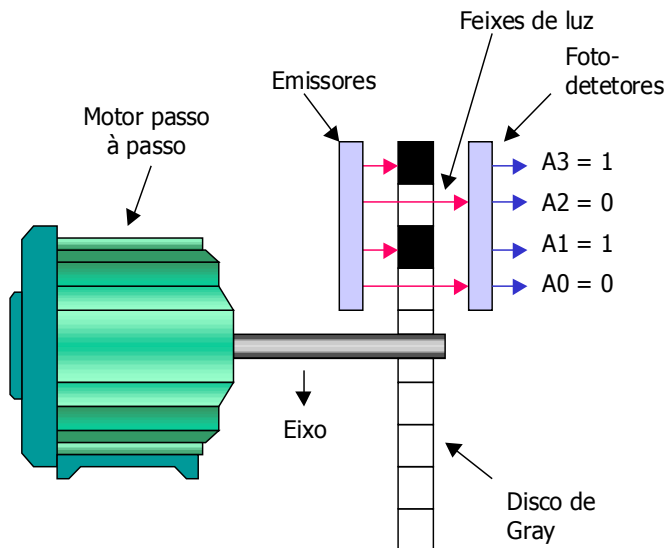
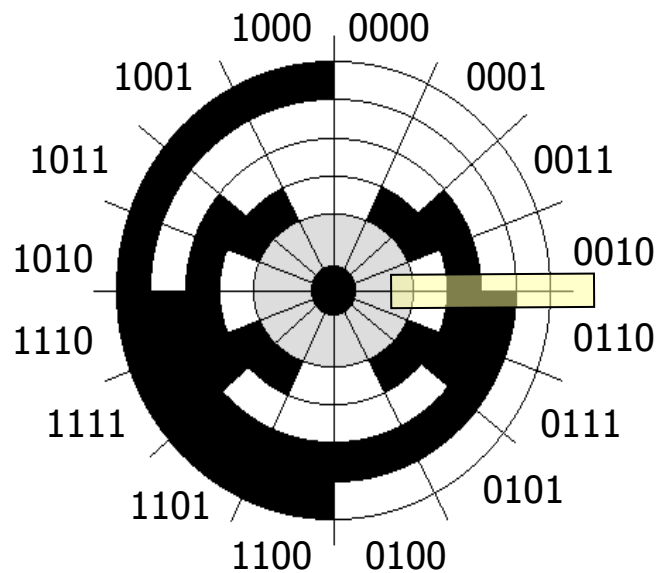
Código Gray

- Código Gray: classe especial de códigos de "distância um".

"Distância um": diferença de um único bit com a palavra código próximo ou adjacente independente da direção.

<u>Código gray</u>	<u>decimal</u>
000	0
001	1
011	2
010	3
110	4
111	5
101	6
100	7

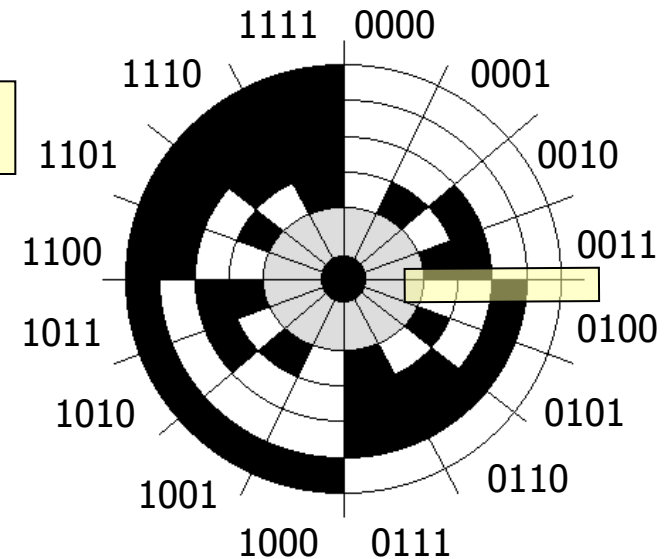
Disco Gray:



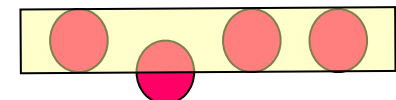
Repare a seqüência:

Ref	Gray	Dec	Bin
0	0000	0	0000
1	0001	1	0001
2	0011	3	0010
3	0010	2	0011
4	0110	6	0100
5	0111	7	0101
6	0101	5	0110
7	0100	6	0111
8	1100	12	1000
9	1101	13	1001
10	1111	15	1010
11	1110	14	1011
12	1010	10	1100
13	1011	11	1101
14	1001	9	1110
15	1000	8	1111

Disco Binário:



Note o **código Gray**: entre uma variação e outra do código, **somente um bit se altera**. Evita erros de leitura se alguns dos foto-detecetores se encontra desalinhado frente a seus "companheiros":



Exemplo de Uso de Código Gray

- Encoder absoluto: sensor de posição angular!

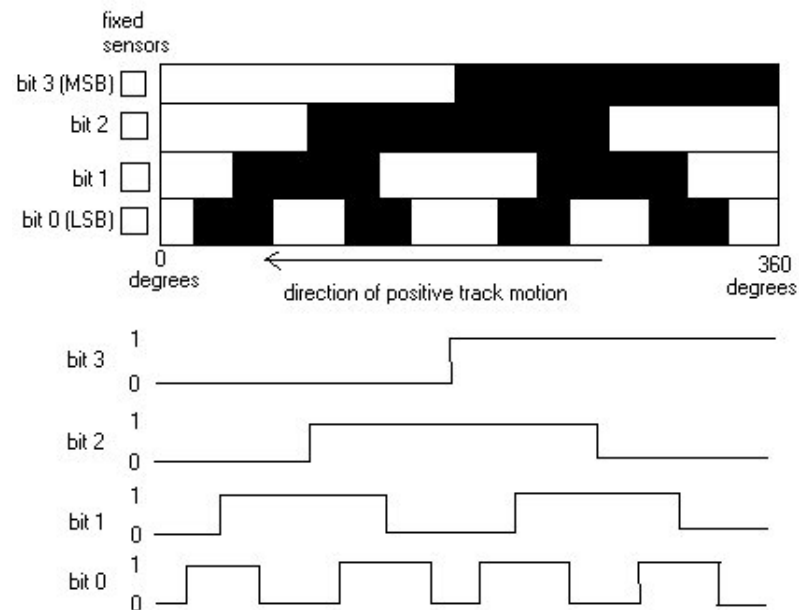
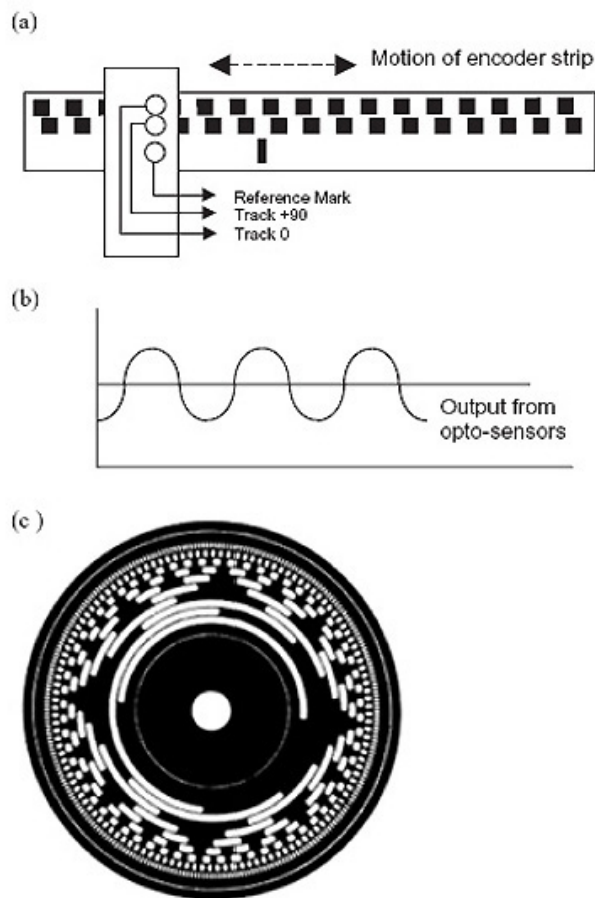
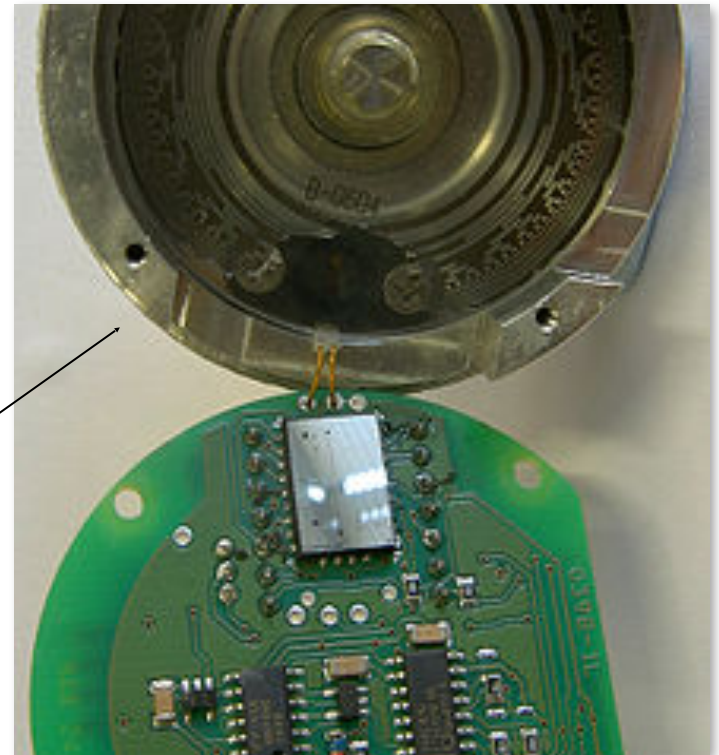
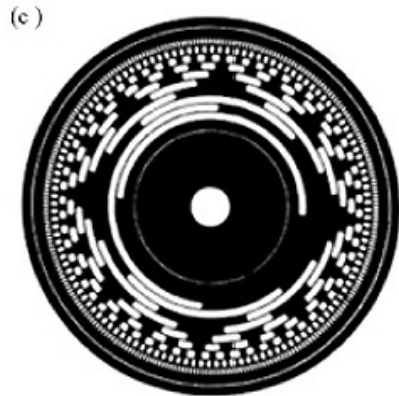
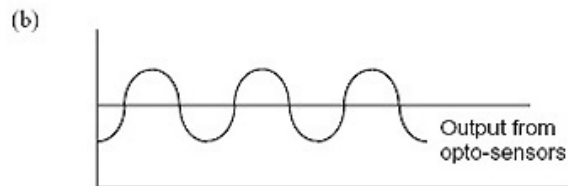
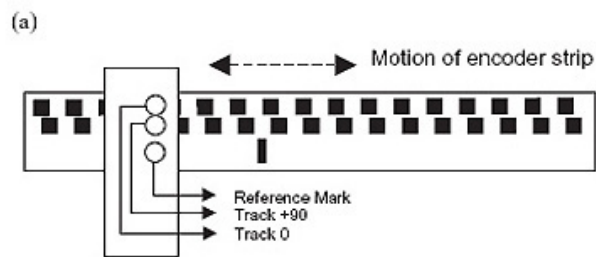


Fig 2. 4-Bit gray code absolute encoder disk track patterns

Exemplo de Uso de Código Gray

- Encoder absoluto: sensor de posição angular!



Um codificador rotativo absoluto de código Gray com 13 trilhas. No topo podem ser vistos o invólucro, o disco óptico e a fonte de luz; na parte inferior pode ser visto o elemento sensor e os componentes de suporte.

Código Binário:: Não é Encoder Absoluto!

Seq. **Gray** (4-bits)

×

Seq. **Binária** (4-bits)

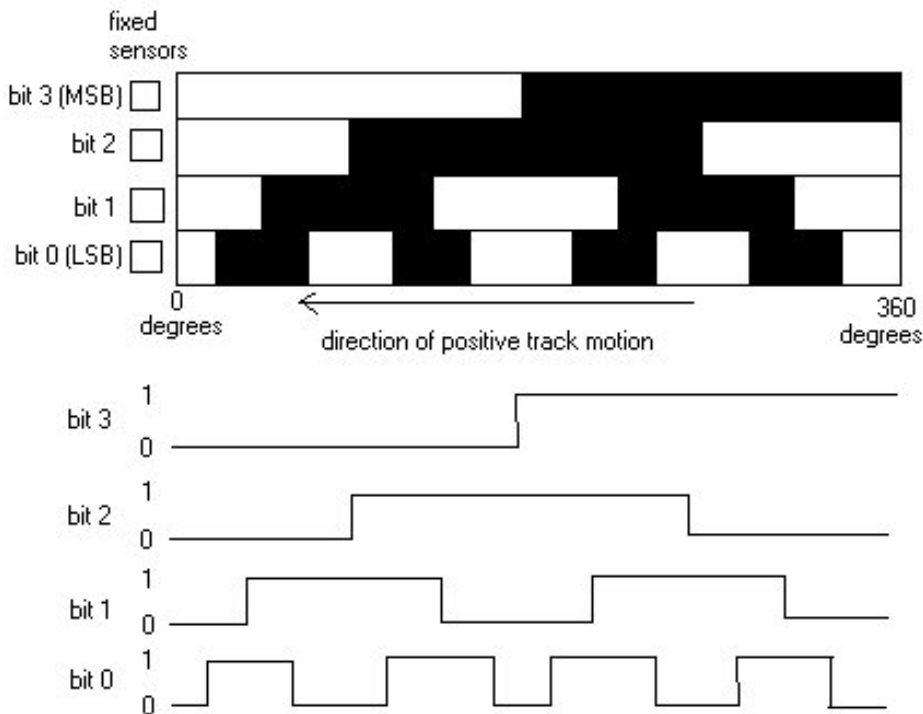


Fig 2. 4-Bit gray code absolute encoder disk track patterns

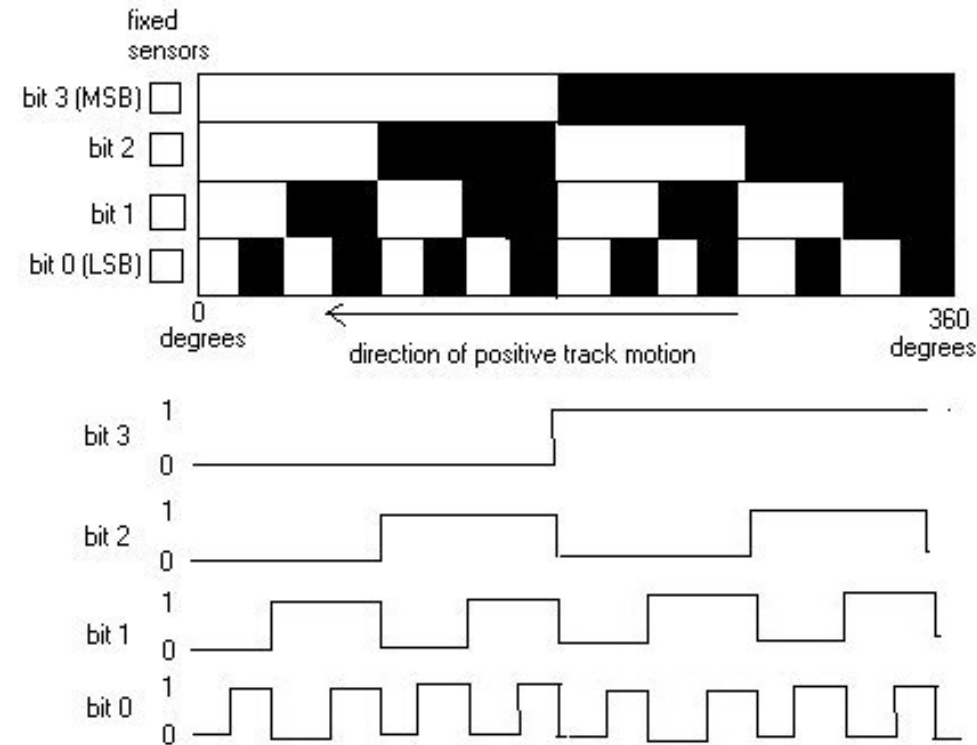


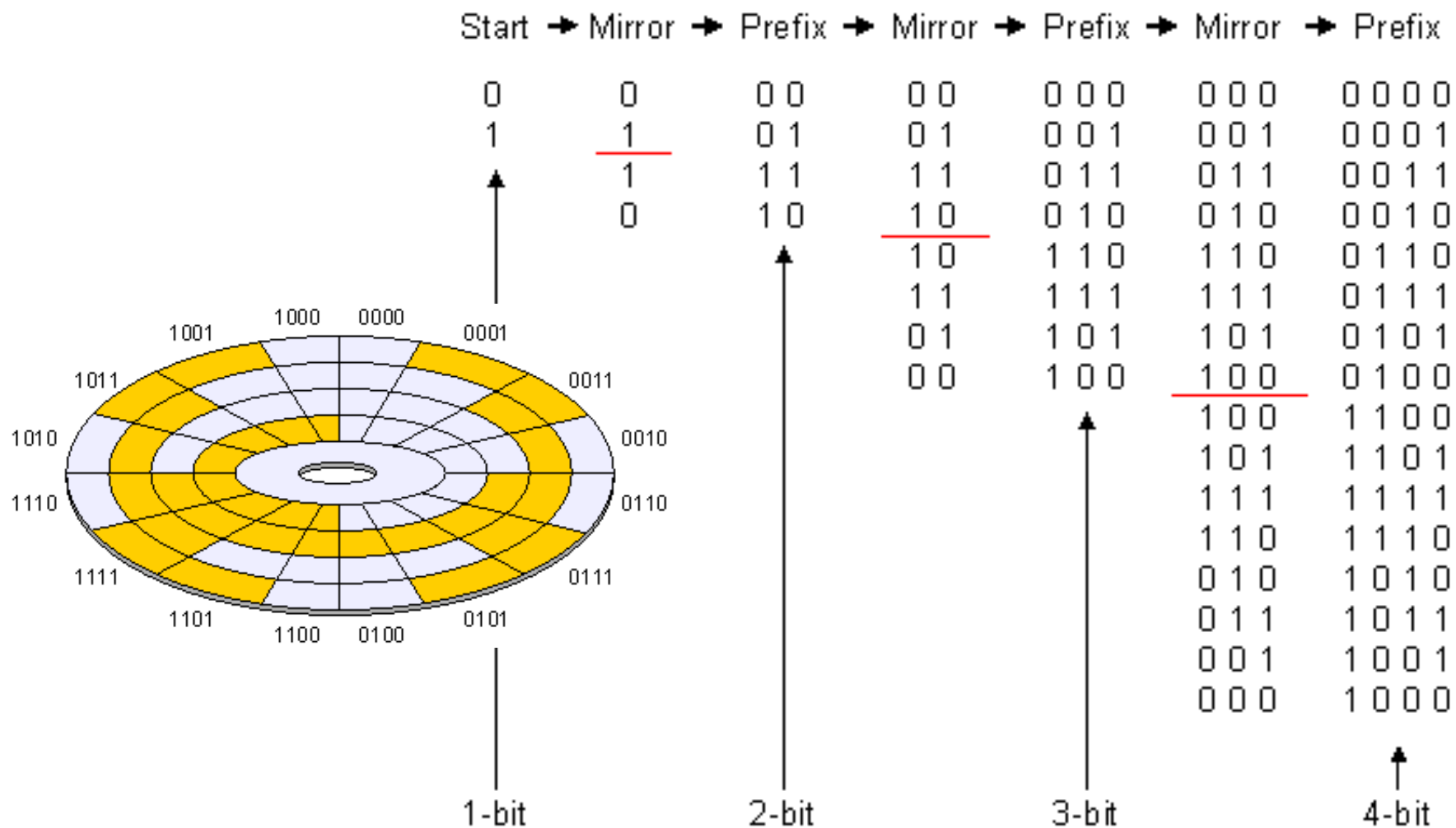
Fig 3. 4-Bit binary code absolute encoder disk track patterns

Menos erros

A cada passo: 1 bit apenas varia de estado!

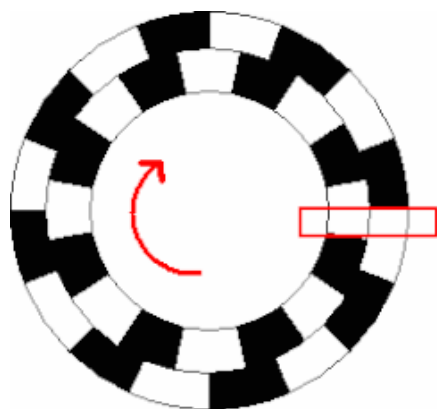
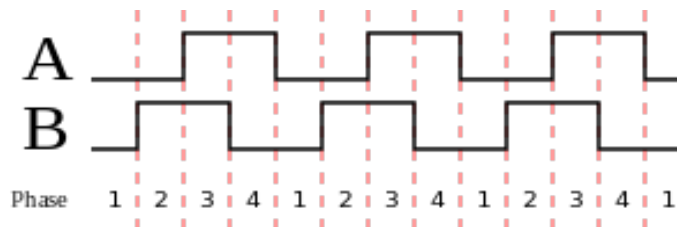
“Saltos grandes” => + erros

Sequência do Código Gray:

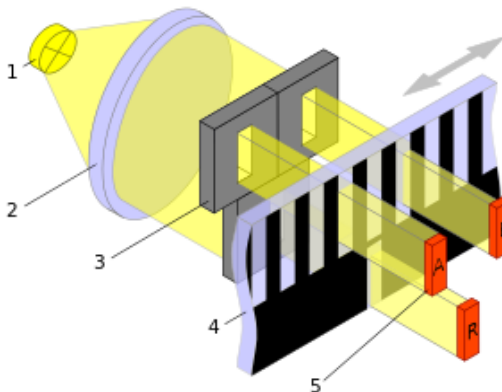
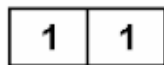


Encoder Absoluto x Encoder Incremental ou Relativo

Duas ondas quadradas em quadratura. A direção do movimento é indicada pelo sinal da diferença de fase A-B que, neste caso, é negativa porque A segue B.



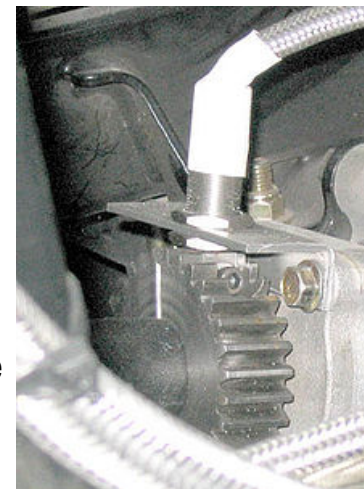
Codificador rotativo, com estados dos sinais A/B correspondentes mostrados à direita



Codificador linear; o sinal R (index) indica que o codificador está localizado em sua posição de referência.



Codificador de quadratura de efeito Hall, detecção de dentes de engrenagem no eixo de transmissão de um veículo.



Código ASCII

32:		33:	!	34:	"	35:	#	36:	\$	37:	%	38:	&	39:	'	40:	(	41:	)
42:	*	43:	+	44:	,	45:	-	46:	.	47:	/	48:	0	49:	1	50:	2	51:	3
52:	4	53:	5	54:	6	55:	7	56:	8	57:	9	58:	:	59:	;	60:	<	61:	=
62:	>	63:	?	64:	@	65:	A	66:	B	67:	C	68:	D	69:	E	70:	F	71:	G
72:	H	73:	I	74:	J	75:	K	76:	L	77:	M	78:	N	79:	O	80:	P	81:	Q
82:	R	83:	S	84:	T	85:	U	86:	V	87:	W	88:	X	89:	Y	90:	Z	91:	[
92:	\	93:	]	94:	^	95:	_	96:	`	97:	a	98:	b	99:	c	100:	d	101:	e
102:	f	103:	g	104:	h	105:	i	106:	j	107:	k	108:	l	109:	m	110:	n	111:	o
112:	p	113:	q	114:	r	115:	s	116:	t	117:	u	118:	v	119:	w	120:	x	121:	y
122:	z	123:	{	124:		125:	}	126:	~	127:	À	128:	Ç	129:	Ü	130:	é	131:	â
132:	ä	133:	à	134:	ã	135:	ç	136:	ê	137:	ë	138:	è	139:	ï	140:	î	141:	ì
142:	Ä	143:	Å	144:	É	145:	æ	146:	Æ	147:	ô	148:	ö	149:	ò	150:	û	151:	ù
152:	ÿ	153:	ö	154:	ü	155:	ø	156:	£	157:	Ø	158:	×	159:	f	160:	á	161:	í
162:	ó	163:	ú	164:	ñ	165:	Ñ	166:	à	167:	ó	168:	¿	169:	Ø	170:	¬	171:	½
172:	¼	173:	ì	174:	«	175:	»	176:	⋮	177:	⋮	178:	⋮	179:		180:	†	181:	Â
182:	Â	183:	À	184:	Ø	185:	‡	186:		187:	¶	188:	¶	189:	¢	190:	¥	191:	Γ
192:	Ł	193:	±	194:	‡	195:	‡	196:	—	197:	†	198:	ã	199:	Ã	200:	ℓ	201:	ƒ
202:	≡	203:	⌣	204:	‡	205:	=	206:	†	207:	¤	208:	§	209:	Ð	210:	Ê	211:	Ë
212:	È	213:	■	214:	Í	215:	Î	216:	Ï	217:	Ɔ	218:	Ɔ	219:	■	220:	■	221:	┆
222:	Ì	223:	■	224:	Ó	225:	ß	226:	Ô	227:	Ò	228:	Õ	229:	Ö	230:	μ	231:	Ɔ
232:	Ɔ	233:	Ú	234:	Û	235:	Ü	236:	Ý	237:	Ÿ	238:	ˆ	239:	ˆ	240:	-	241:	±
242:	=	243:	¾	244:	¶	245:	§	246:	÷	247:	˘	248:	◊	249:	..	250:	.	251:	'
252:	³	253:	²	254:	▪	255:													

ascii.cpp (gera tabela ASCII):

```
// Tabela ASCII
// Fernando Passsold, 05/09/2001

#include <stdio.h>
// #include <stdlib.h>
void main() {
    int codigo = 32, coluna=1, linha=1, key, aux;

    for (codigo=32; codigo<256; codigo++) {
        printf("%3d: ",codigo);
        fputc(codigo);
        coluna++;
        if (coluna<11){
            printf(" ");
        }
        else {
            printf("\n");
            coluna=1;
            linha++;
            if (linha>24){
                // espera uma tecla ser apertada
                while ( (key = getchar()) != '\n' )
                    printf("%c",key);
                linha=1;
            }
        }
    }
}
```

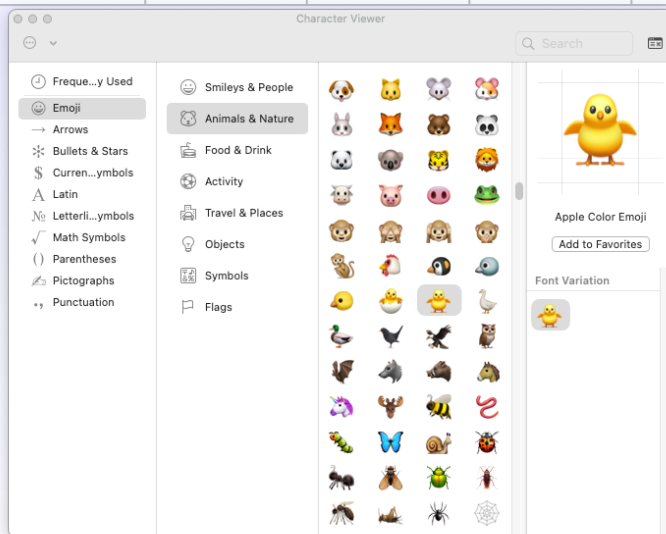
Código UTF-8:

- UTF-8 é uma codificação de caracteres de largura variável usada para comunicação eletrônica. Definido pelo padrão Unicode, o nome é derivado do formato de transformação Unicode (ou conjunto de caracteres codificados universal) - 8 bits.
- O UTF-8 é capaz de codificar todos os 1.112.064 códigos de caracteres válidos em Unicode usando um a quatro bytes (8 bits). Os pontos de código com valores numéricos mais baixos, que tendem a ocorrer com mais frequência, são codificados usando menos bytes. Ele foi projetado para compatibilidade com versões anteriores de ASCII: os primeiros 128 (2^7) caracteres de Unicode, correspondem com o código ASCII, são codificados usando um único byte com o mesmo valor binário de ASCII, de modo que um texto ASCII válido é também um código UTF-8 válido. Os bytes ASCII não conseguem codificar caracteres não ASCII do UTF-8, mas UTF-8 é seguro para uso na maioria das linguagens de programação e de documento que interpretam caracteres ASCII de uma maneira especial, como "/" (barra) em nomes de arquivos, "\" (barra invertida) em sequências de escape e "%" em comandos "printf".
- Os primeiros 128 caracteres (US-ASCII) precisam de um byte. Os próximos 1.920 caracteres precisam de dois bytes para codificar, o que abrange o restante de quase todos os alfabetos de escrita latina e também os alfabetos grego, cirílico, copta, armênio, hebraico, árabe, siríaco, Thaana e N'Ko, bem como os alfabetos diacrítico combinado Marcas. Três bytes são necessários para caracteres no resto do Plano Multilíngue Básico, que contém virtualmente todos os caracteres de uso comum, incluindo a maioria dos caracteres chineses, japoneses e coreanos. Quatro bytes são necessários para caracteres em outros planos de Unicode, que incluem caracteres CJK menos comuns, vários scripts históricos, símbolos matemáticos e emoji (símbolos pictográficos).

Código UTF-8:

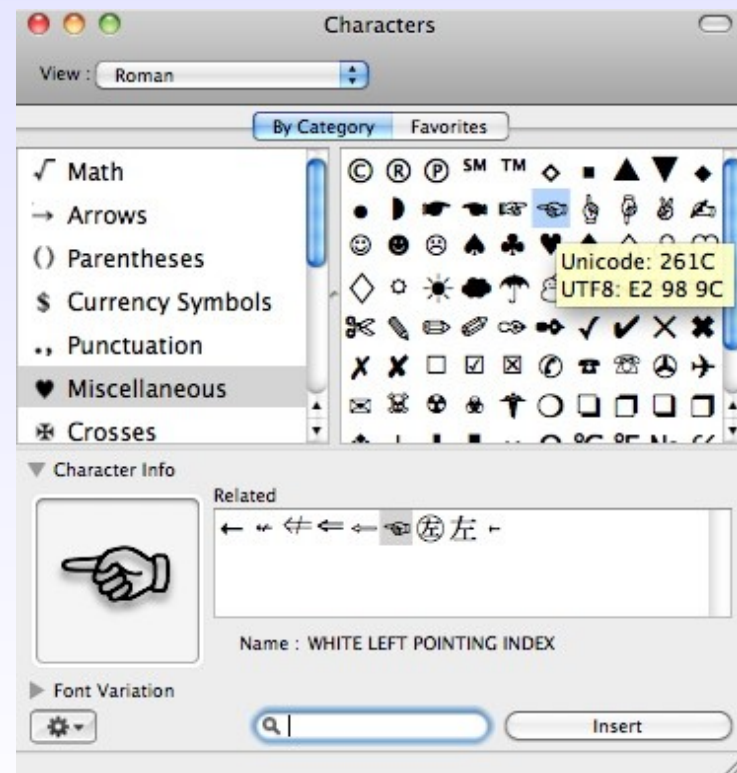
- Exemplo: o código Unicode para "€" é U+20AC.

First code point	Last code point	Byte 1	Byte 2	Byte 3	Byte 4
U+0000	U+007F	0xxxxxxx			
U+0080	U+07FF	110xxxxx	10xxxxxx		
U+0800	U+FFFF	1110xxxx	10xxxxxx	10xxxxxx	
U+10000	U+10FFFF	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx



standing chick

Unicode: U+1F425, UTF-8: F0 9F 90 A5



Resumo Sistemas de Numeração

Decimal	Binário	Hexadecimal	Octal
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	8	10
9	1001	9	11
10	1010	A	12
11	1011	B	13
12	1100	C	14
13	1101	D	15
14	1110	E	16
15	1111	F	17
16	10000	10	20
17	10001	11	21

Prefixos e notações de grandezas no mundo digital

- “K” = Kilo = $2^{10} = 1024$
- “M” = Mega = $1K \cdot 1K = 2^{10} \cdot 2^{10} = 2^{20} = 1.048.576$
- “G” = Giga = $2^{30} = 1.073.741.824$
- “T” = Terra = $2^{40} = 1.099.511.627.776$
- “P” = Penta = 2^{50}

Curiosidades... (Problema)



Um mini-computador Apple IIe do início de 1983, era capaz de endereçar até 64 Kbytes de RAM. Pergunta-se:

- a) Quantas linhas de endereçamento este equipamento (sua CPU, um μP 6502, de 8-bits @ 1,023 MHz) possui?
- b) Este equipamento era capaz de cobrir que faixa de endereçamento, em hexadecimal?
- c) Os primeiros 16Kbytes eram reservados para sua memória ROM (Read Only Memory) que continha um sistema operacional básico e interpretador da linguagem BASIC. Esta porção de memória corresponde à que faixa de endereçamento? E isto significa que os dados e outros programas ficavam armazenados à partir de que posição de memória?

Apple IIe:

64Kbytes ➡ Linhas endereçamento? Faixa?



Resolução:

“64K” significa que era capaz de endereçar até $64 \cdot 2^{10} = 65.536$ posições de memória.

Para endereçar esta quantidade de memória, se faz necessário um barramento de endereços constituído por:

$65.536 = 64 \cdot 2^{10} = 2^6 \cdot 2^{10} = 2^{16}$, ou seja, 16 linhas de endereçamento, iniciando de A_0 até A_{15} .

O primeiro endereço corresponde à posição: 0 (zero).

O último endereço (posição) corresponde à $64K - 1 = 65.536 - 1 = 65.535$; em hexadecimal:

- Com $64K = 2^6 \cdot 2^{10} = 2^{16} \Rightarrow$ bit da posição 16 ativado.
Lembramos que cada 4-bits $\Rightarrow$ 1 algarismo hexadecimal. Teremos algo como:

A_{16}	A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	Endereço
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	10000H

- Mas o equipamento possui 16 linhas de endereçamento (não 17), então a última posição de memória corresponde (em hexadecimal) à:

A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	Endereço
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	FFFFH

Assim, este equipamento era capaz de endereçar da posição: 0000H até FFFFH.

Apple IIe $\Rightarrow$ até 64Kbytes
ROM: 16K

$\Rightarrow$ Faixa endereçamento?

Resolução:

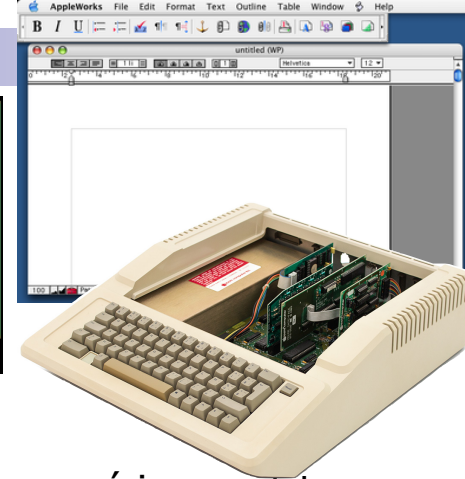
Note que era comum dividir a memória em “segmentos” (blocos).

Neste caso, um bloco de 16K, significava fazer: $64K \div 16K = 4$, ou seja, dividir a memória completa em 4 blocos de 16K.

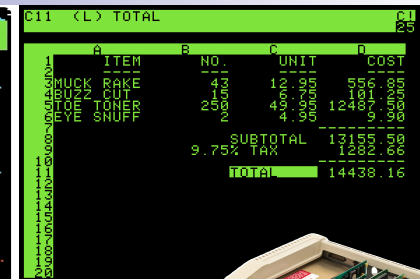
$16K = 2^4 \cdot 2^{10} = 2^{14}$, ou seja, usa-se 14-bits para endereçar uma posição dentro de cada bloco de 16K. Em Hexadecimal:

Bloco 16K	A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	Endereço	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0000H	ROM: reservada para o sistema
	0	0	:	:	:	:	:	:	:	:	:	:	:	:	:	:		
	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	3FFFH	
1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	4000H	RAM: programas e dados do usuário
	0	1	:	:	:	:	:	:	:	:	:	:	:	:	:	:		
	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	7FFFH	
2	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	8000H	
	1	0	:	:	:	:	:	:	:	:	:	:	:	:	:	:		
	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	BFFFH	
3	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	C000H	
	1	1	:	:	:	:	:	:	:	:	:	:	:	:	:	:		
	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	FFFFH	

Então os primeiros 16K são cobertos pela faixa de endereçamento: 0000H até 3FFFH.



Apple IIe $\Rightarrow$ at
ROM: 16K
 $\Rightarrow$ Faixa ender



Resolução:

Note que era comum dividir a memória em “segmentos” (blocos).

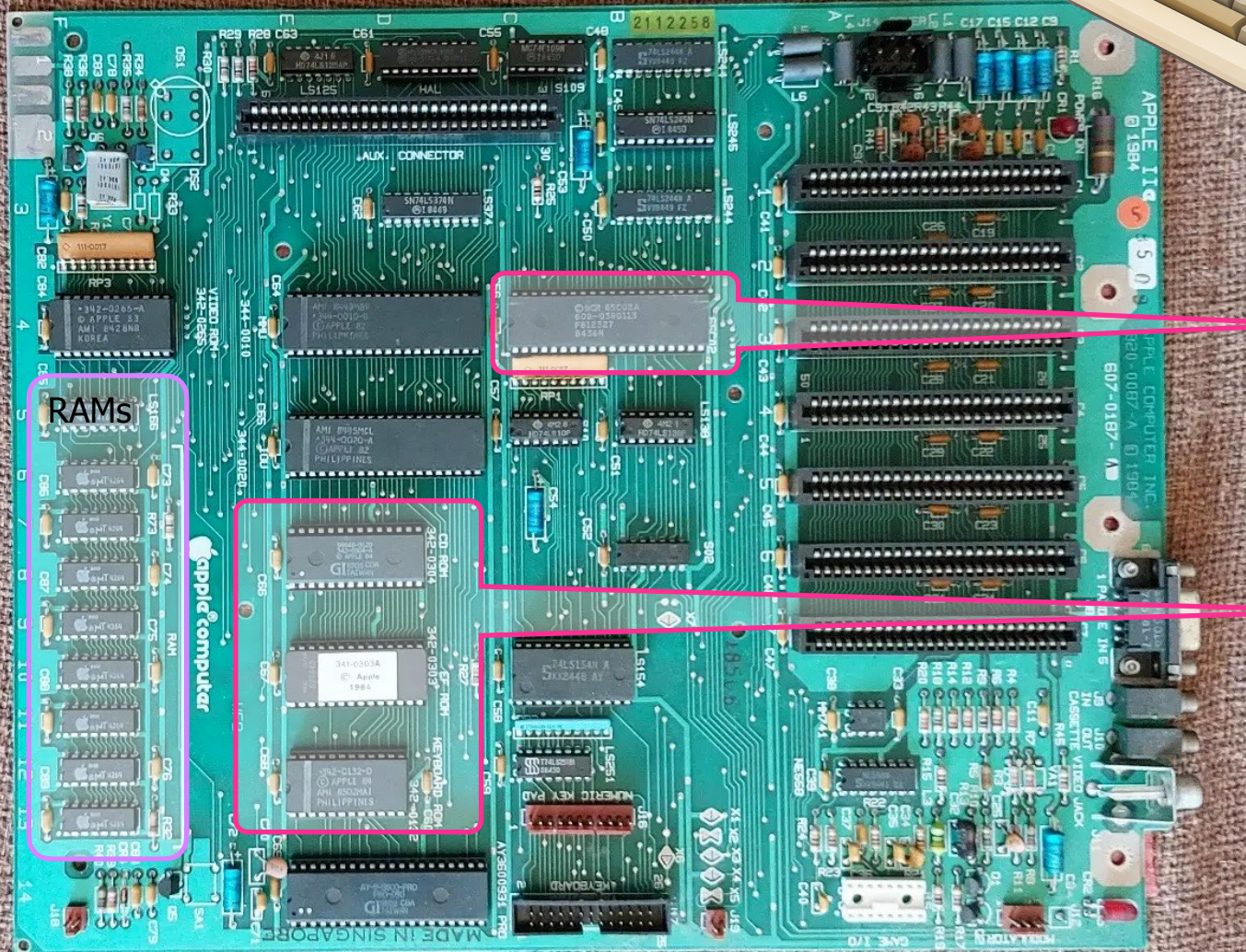
Neste caso, um bloco de 16K, significava fazer: $64K \div 16K = 4$, ou seja, dividir a memória completa em 4 blocos de 16K.

$16K = 2^4 \cdot 2^{10} = 2^{14}$, ou seja, usa-se 14-bits para endereçar uma posição dentro de cada bloco de 16K. Em Hexadecimal:

Bloco 16K	A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	Endereço	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0000H	ROM: reservada para o sistema
	0	0	:	:	:	:	:	:	:	:	:	:	:	:	:	:		
	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	3FFFH	
1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	4000H	RAM: programas e dados do usuário
	0	1	:	:	:	:	:	:	:	:	:	:	:	:	:	:		
	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	7FFFH	
2	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	8000H	
	1	0	:	:	:	:	:	:	:	:	:	:	:	:	:	:		
	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	BFFFH	
3	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	C000H	
	1	1	:	:	:	:	:	:	:	:	:	:	:	:	:	:		
	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	FFFFH	

Então os primeiros 16K são cobertos pela faixa de endereçamento: 0000H até 3FFFH.

Apple IIe

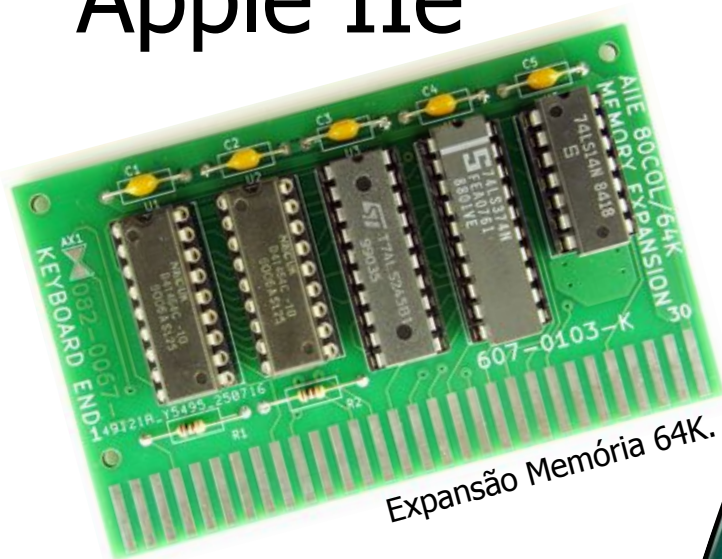


uP: 65C02

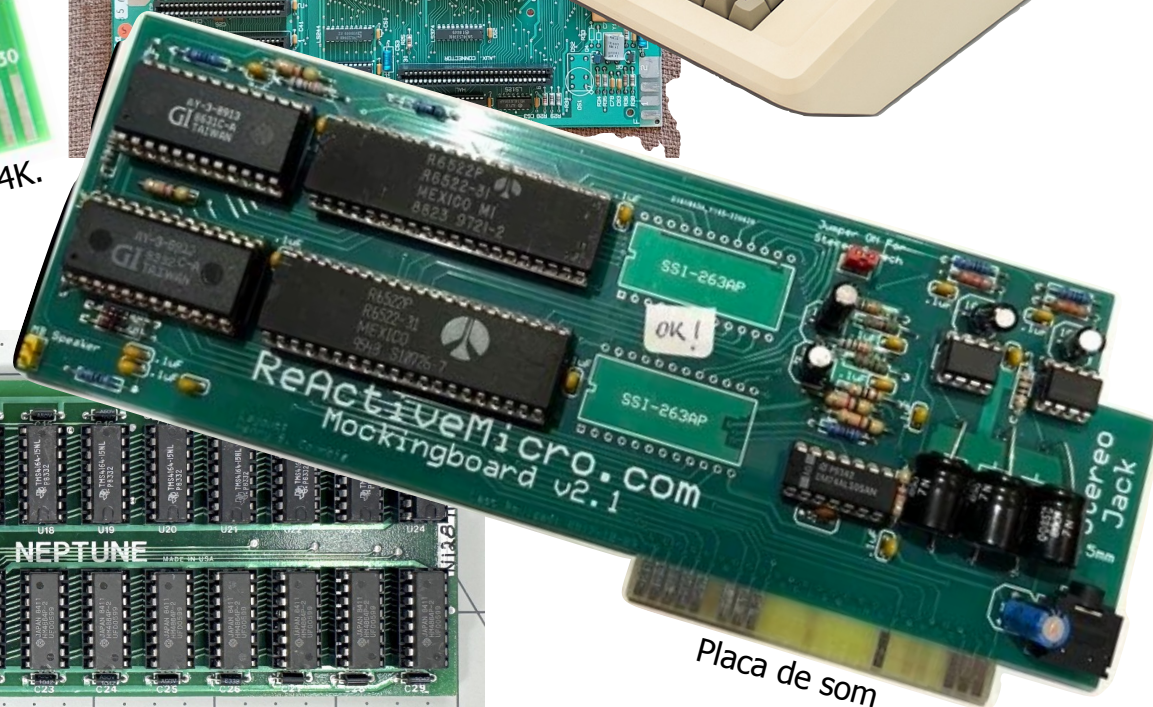
ROMs

Placa mãe.

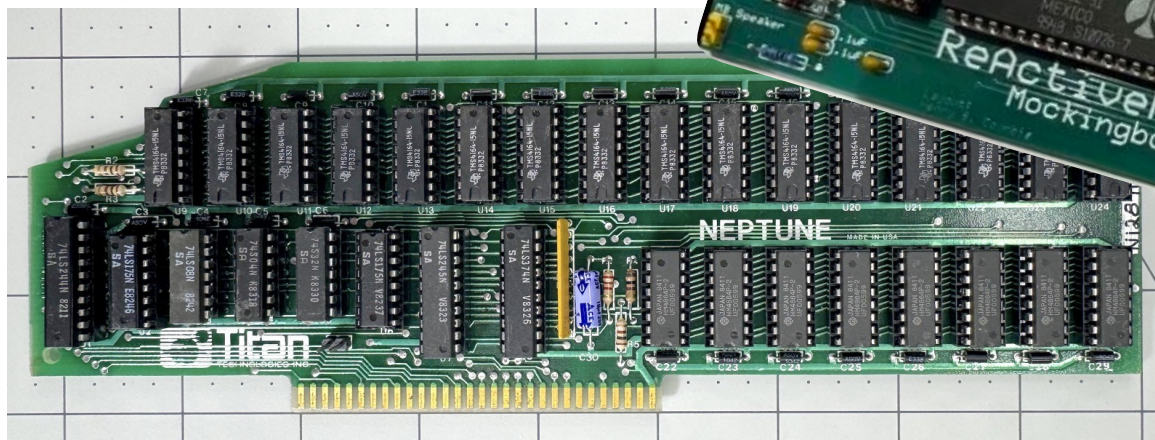
Apple IIe



Expansão Memória 64K.



Placa de som



Expansão Memória 192K.